

Cracking Open the OpenAI (Python) API

A complete beginner-friendly introduction with example code



Shawhin Talebi · Follow
Published in Towards Data Science · 11 min read · 17 hours ago

👍 15

💬 2





Photo by [Martin Sanchez](#) on [Unsplash](#)

This is the 2nd article in a [series](#) on using Large Language Models (LLMs) in practice. Here I present a beginner-friendly introduction to the OpenAI API. This allows you to go beyond restrictive chat interfaces like ChatGPT and to get more out of LLMs for your unique use cases. Python example code is provided below and at the [GitHub repository](#).

Table of Contents:

1. What's an API?
2. OpenAI's (Python) API
3. Getting Started (4 Steps)
4. Example Code

In the [first article](#) of this series, I described **Prompt Engineering** as the **most accessible way to use LLMs** in practice. The easiest (and most popular) way to do this is via tools like ChatGPT, which provide an intuitive, no-cost, and no-code way to interact with an LLM.

A Practical Introduction to LLMs
3 levels of using LLMs in practice
[towardsdatascience.com](#)

However, this ease of use comes at a cost. Namely, the chat UI is restrictive and does not translate well to many practical use cases e.g. building your own customer support bot, real-time sentiment analysis of customer reviews, etc.

In these cases, we can take Prompt Engineering one step further and interact with LLMs *programmatically*. One way we can do this is via an API.

1) What's an API?

An **application programming interface (API)** allows you to interact with a remote application programmatically. While this might sound technical and scary, the idea is super simple. Consider the following analogy.

Imagine you have an intense craving for the **pupusas** you ate during that summer in El Salvador. Unfortunately, you're back at home and don't know where to find good Salvadoran food. Lucky for you, however, you have a super-foodie friend that knows every restaurant in town.

So, you send your friend the text.

"Any good pupusa spots in town?"

Then, a couple of minutes later, you get the response.

"Yes! Flavors of El Salvador has the best pupusas!"

While this may seem irrelevant to APIs, this is essentially how they work. You send a **request** to a remote application i.e. text your super-foodie friend. Then, the remote application sends back a **response** i.e. the text back from your friend.

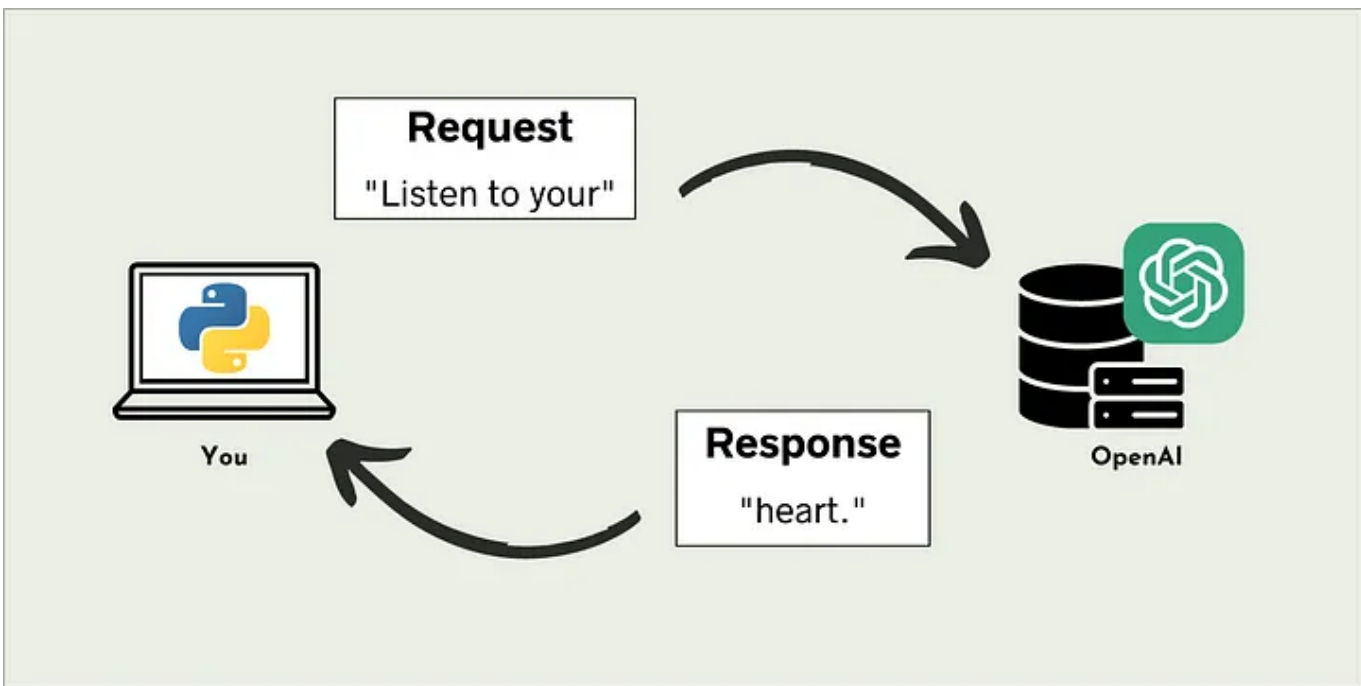


A visual analogy of how APIs work. Image by author.

The difference between an API and the above analogy is instead of sending the request with your phone's texting app, you use your favorite programming language e.g. Python, JavaScript, Ruby, Java, etc. This is great if you are developing software where some external information is required because the information retrieval can be automated.

2) OpenAI's (Python) API

We can use APIs to interact with Large Language Models. A popular one is OpenAI's API, where instead of typing prompts into the ChatGPT web interface, you can send them to and from OpenAI using Python.



Visualization of how API calls to OpenAI works. Image by author.

This gives virtually anyone access to state-of-the-art LLMs (and other ML models) without having to provision the computational resources needed to run them. The downside, of course, is OpenAI doesn't do this as a charity. Each API call costs money, but more on that in a bit.

Some **notable features** of the API (not available with ChatGPT) are listed below.

- **Customizable system message** (this is set to something like *"I am*

architecture. My knowledge is based on information available up until September 2021. Today's date is July 13, 2023." for ChatGPT)

- **Adjust input parameters** such as maximum response length, number of responses, and temperature (i.e. the “randomness” of the response).
- Include **images** and **other file types** in prompts
- Extract helpful word **embeddings** for downstream tasks
- **Input audio** for transcription or translation
- Model **fine-tuning** functionality

The OpenAI API has **several models** from which to choose. The *best* model to pick will depend on your particular use case. Below is a list of the current models available [1].

OpenAI API Models
(as of July 2023)

- **GPT-4:** OpenAI's latest large multimodal model. Significant improvement in "complex reasoning tasks" compared to GPT-3.5 models. Optimized for chat. *Input:* text/image. *Output:* text.)
- **GPT-3.5-turbo:** Model running the free version of ChatGPT. Optimized for chat. *Input:* text. *Output:* text.)
- **Whisper:** General purpose speech recognition model. Can do transcription, translation, and detect language. Model is open source. *Input:* audio. *Output:* text.)
- **Embeddings:** Generates (useful) vector representation of text which can be used for downstream tasks e.g. text similarity. *Input:* text. *Output:* numerical vector.)
- **GPT-3 models:** Original ChatGPT models. Less powerful than GPT-3.5 models, however, GPT-3 models are cheaper. *Input:* text. *Output:* text.)
- **Moderation:** A fine-tuned model to detect NSFW content. *Input:* text. *Output:* Category (e.g. hate, harassment, ...)
- **DALL-E:** Creates realistic images based on user prompts. *Input:* text or image. *Output:* image.)

List of available models via the OpenAI API as of Jul 2023. Image by author. [1]

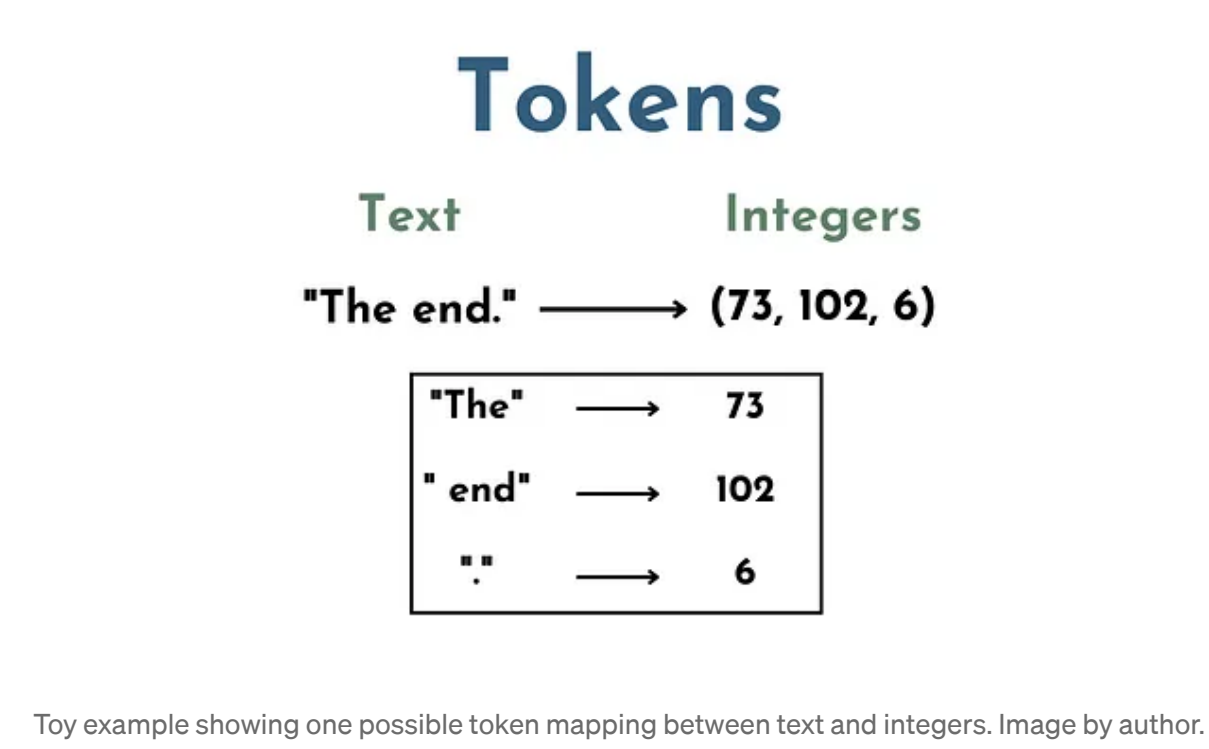
Note: Each item listed above is accompanied by a set of models which vary in size and cost. Check [documentation](#) for the most recent information.

Pricing & Tokens

While the OpenAI API gives developers easy access to SOTA ML models, one obvious downside is that it **costs money**. Pricing is done on a per-token basis (no, I don't mean NFTs or something you use at the arcade).

Tokens, in the context of LLMs, are essentially a **set of numbers representing a set of words and characters**. For example, “The” could be a token, “ end” (with the space) could be another, and “.” another.

Thus, the text “The End.” would consist of 3 tokens say (73, 102, 6).



This is a critical step because LLMs (i.e. **neural networks**) **do not “understand” text directly**. The text must be converted into a numerical representation so that the model can perform mathematical operations on the input. Hence, the tokenization step.

The price of an API call depends on the number of tokens used in the prompt and the model being prompted. The price per model is available on [OpenAI's website](#).

3) Getting Started (4 Steps)

Now that we have a basic understanding of the OpenAI API let's see how to use it. Before we can start coding, we need to set up four things.

3.1) Make an Account (you get a \$5 API credit for 1st three months)

1. To make an account go to the [OpenAI API Overview page](#), and click “Sign Up” in the top right corner
2. *Note* — If you've used ChatGPT before, then you probably already have an OpenAI account. If so, click “Log in”

3.2) Add Payment Method

1. If your account is more than 3 months old or the free \$5 API credit is not enough for you, you will need to add a payment method before making API calls.
2. Click your profile image and select the manage account option.
3. Then add a payment method by clicking the “Billing” tab and then “Payment methods”.

3.3) Set Usage Limits

1. Next, I recommend setting usage limits so that you **avoid being billed more than you budget for**.
2. To do this, go to the “Usage limits” under the “Billing” tab. Here you can set a “Soft” and “Hard” limit.
3. If you hit your monthly **soft limit**, OpenAI will send you an **email notification**.
4. If you hit your **hard limit**, any additional API requests will be denied (thus, you won't be charged more than this).

3.4) Get API Secret Key

1. Click on “View API keys”
2. If this is your first time, you will need to make a new secret key. To do this, click “Create new secret key”
3. Next, you can give your key a custom name. Here I used “my-first-key”.
4. Then, click “Create secret key”

4) Example Code: Chat Completion API

With all the setup done, we are (finally) ready to make our first API call. Here we will use the [openai Python library](#), which makes integrating OpenAI's models into your Python code super easy. You can download the package via [pip](#). The below example code (and bonus code) is available on the [Github repo](#) for this article.

A quick note on Completions API deprecations — OpenAI is moving away from the freeform prompt paradigm and toward chat-based API calls. According to a blog from OpenAI, the chat-based paradigm provides better responses, given its structured prompt interface, compared to the previous paradigm [2].

While older OpenAI (GPT-3) models are still available via the “freeform” paradigm, the more recent (and powerful) models (i.e. GPT-3.5-turbo and GPT-4) are only available via chat-based calls.

Let's start with a super simple API call. Here we will pass **two inputs** into the `openai.ChatCompletions.create()` method i.e. **model** and **messages**.

1. **model** — defines the name of the language model we want to use (we can choose from the models listed earlier in the article.)
2. **messages** — sets the “preceding” chat dialogue as a list of dictionaries. The dictionaries have two key-value pairs (e.g. {“role”: “user”, “content”: “Listen to your”}). First, “role” defines *who is talking* (e.g. “role”: “user”). This can either be the “user”, “assistant”, or “system”. Second, “content” defines *what the role is saying* (e.g. “content”:

“Listen to your”). While this may feel more restrictive than a freeform prompt interface, we can get creative with input messages to optimize responses for a particular use case (more on this later).

This is what our first API call looks like in Python.

```
import openai
from sk import my_sk # importing secret key from external file
import time

# imported secret key (or just copy-paste it here)
openai.api_key = my_sk

# create a chat completion
chat_completion = openai.ChatCompletion.create(model="gpt-3.5-turbo",
                                              messages=[{"role": "User", "content": "Listen to your"}])
```

The API response is stored in the *chat_completion* variable. Printing *chat_completion*, we see that it is like a dictionary consisting of 6 key-value pairs.

```
{'id': 'chatcmpl-7dk1JkF5SDm2422nVRPL9x0QrLhIZ4',
 'object': 'chat.completion',
 'created': 1689706049,
 'model': 'gpt-3.5-turbo-0613',
 'choices': [{'OpenAIObject at 0x7f9d1a862b80': JSON: {
  "index": 0,
  "message": {
    "role": "assistant",
    "content": "heart."
  },
  "finish_reason": "stop"
}]}],
 'usage': {'OpenAIObject at 0x7f9d1a862c70': JSON: {
  "prompt_tokens": 10,
  "completion_tokens": 2,
  "total_tokens": 12
}}}
```

The meaning of each field is listed below.

- ‘Id’ = unique ID for the API response
- ‘Object’ = name of API object that sent the response
- ‘Created’ = unix timestamp of when the API request was processed
- ‘Model’ = name of the model used
- ‘Choices’ = model response formatted in JSON (i.e. dictionary-like)
- ‘Usage’ = token count meta-data formatted in JSON (i.e. dictionary-like)

However, the main thing we care about here is the ‘Choices’ field since this is **where the model response is stored**. In this case, we see the “assistant” role responds with the message *“heart.”*

Yay! We made our 1st API call. Now let's start playing with the model input parameters.

max_tokens

First, we can set the **maximum number of tokens** allowed in the model response using the *max_tokens* input parameter. This can be helpful for many reasons depending on the use case. In this case, I just want a one-word response, so I'll set it to 1 token.

```
# setting max number of tokens

# create a chat completion
chat_completion = openai.ChatCompletion.create(model="gpt-3.5-turbo",
                                              messages=[{"role": "User", "content": "Listen to your"}],
                                              max_tokens = 1)

# print the chat completion
print(chat_completion.choices[0].message.content)

"""
Output:
>>> heart
"""
```

n

Next, we can set the **number of responses** we would like to receive from the model. Again, this can be helpful for many reasons depending on the use case. For example, if we want to generate a set of responses from which we can select the one we like best.

```
# setting number of completions

# create a chat completion
chat_completion = openai.ChatCompletion.create(model="gpt-3.5-turbo",
                                              messages=[{"role": "User", "content": "Listen to
                                              max_tokens = 2,
                                              n=5)

# print the chat completion
for i in range(len(chat_completion.choices)):
    print(chat_completion.choices[i].message.content)

"""
Output:
>>> heart.
>>> heart and
>>> heart.
>>>
>>> heart,
>>>
>>> heart,
"""
```

Notice that **not all the completions are identical**. This may be a good thing or a bad thing based on the use case (e.g. creative use cases vs. process automation use cases). Therefore, it can be advantageous to adjust the *diversity* of chat completions for a given prompt.

temperature

It turns out we can do this by tuning the **temperature** parameter. Put simply, this **adjusts the “randomness” of chat completions**. Values for this parameter **range from 0 to 2**, where 0 makes completions more predictable, and 2 makes them less predictable [3].

Conceptually, we can think of temp=0 will default to the most likely next word while temp=2 will enable completions that are relatively unlikely. Let's see what this looks like.

```
# temperature=0

# create a chat completion
chat_completion = openai.ChatCompletion.create(model="gpt-3.5-turbo",
                                              messages=[{"role": "User", "content": "Listen to
                                              max_tokens = 2,
                                              n=5,
                                              temperature=0)

# print the chat completion
for i in range(len(chat_completion.choices)):
    print(chat_completion.choices[i].message.content)

"""
Output:
>>> heart.
>>> heart.
>>> heart.
>>> heart.
>>> heart.
"""
```

As expected, when temp=0, all 5 completions are identical and produce something “very likely.” Now let's see what happens when we **turn up the temperature**.

```
# temperature=2

# create a chat completion
chat_completion = openai.ChatCompletion.create(model="gpt-3.5-turbo",
                                              messages=[{"role": "User", "content": "Listen to
                                              max_tokens = 2,
                                              n=5,
```